INDEX

Name Aayush Kumar ~~Shah~~. Ramijee

Standard IV Sec Section A Roll No. 20

Subject OS lab copy.

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1)	24/02/26	To find second smallest element in an array.		
2)	24/02/26	To find sum of diff diagonals		
3)	3/03/26	To simulate CPU scheduling algorithm		
4)	17/03	Round Robin Algorithm.	8	
5)	24/03	Multi level Queue scheduling.	9	
6)	31/03	C program to simulate Real-Time CPU Scheduling algorithms :- a) Rate - Monotonic b) Earliest-deadline First.	11	
7)	21/04	C program to simulate a) Producer - Consumer problem using Semaphores. b) Dining - Philosphers problem	13	
8)	28/04	Banker's algorithm for dead lock Avoidance.	15	
9)	05/05	To simulate Deadlock detection Algorithm.		

INDEX

Name Aayush Rouniyar

Standard IV Sem Section 4A Roll No. 1BF24CS010.

Subject OS-lab Copy.

SL No.	Date	Title	Page No.	Teacher Sign / Remarks
10)	12/05	~~Write a~~ C program to simulate the following contiguous memory allocation technique: (i) First fit (ii) Worst fit (iii) Best fit		
11)	26/05	C program to simulate page replacement algorithms a) FIFO b) LRU c) Optimal.		10

Week - 1

To find second smallest element in an array.

⇒
```
#include<stdio.h>
int main(){
    int n,i, smallest, secsmallest;
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    if (n<2){
        printf("Array must have atleast 2 elements.\n");
        return 1;
    }
    int arr[n];
    printf("Enter %d elements: \n", n);
    for(i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    if (arr[0] < arr[1]){
        smallest = arr[0];
        secSmallest = arr[0];
    }
    for (i=2; i<n; i++){
        if (arr[i] < smallest){
            secSmalest = smallest;
            smallest = arr[i];
        }
        else if (arr[i] < secsmallest && arr[i] != smallest){
            secSmallest = arr[i];
        }
    }
}
```

```
    if (smallest == secsmallest){
        printf("There is no second smallest element");
    }
    else{
    printf("The second smallest element is %d\n",
                secsmallest);
    }
    return 0;
}
```

Output:-

Enter the number of elements in the array : 1.
Array must have atleast 2 element.

Enter number of elements in the array: 3
Enter 3 elements: 2
2
2
There is no second smallest element.

Enter number of elements in the array: 4
Enter 4 elements: 3
2
1
4
The second smallest element is 2.

To find the sum of the left diagonals of a matrix.

```
#include<stdio.h>
int main(){
    int n,i,j,sum=0;
    printf("Enter the size of the square matrix: ");
    scanf("%d",&n);

    int matrix[n][n];

    printf("Enter %d x %d matrix elements:\n",n,n);
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&matrix[i][j]);
        }
    }
    for(i=0;i<n;i++){
        sum+=matrix[i][i];
    }
    printf("Sum of left diagonal (primary diagonal
        elements): %d\n",sum);
    return 0;
}
```

Output:
Enter the size of the square matrix: 2
Enter 2x2 matrix elements
3
5
2
7
Sum of left diagonal (primary diagonal) elements: 10

Week-2

To simulate CPU scheduling algorithm.

1) FCFS
2) SJF.

Code.

⇒
```
#include <stdio.h>
#define MAX 20.

void FCFS (int n, int at[], int bt[]){
    int wt[MAX], tat[MAX];
    int time=0;
    float total-wt=0, total-tat=0;

    for (int i=0; i<n; i++){
        if (time<at[i]){
            time=at[i]; }
        rt[i]= time-at[i];
        wt[i]= time-at[i];
        wt[i]= rt[i];
        time+=bt[i]
        ct[i]= time;
        tat[i]= ct[i]-at[i];

        total-wt += wt[i];
        total-tat += tat[i];
        total-rt += rt[i];
    }
    printf("\n --- FCFS --- \n");
    printf("PID \tAT\t BT\t CT\t TAT\t WT\t RT\n");
```

```
    for (int i=0; i<n; i++){
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }
}

void sjf-nonpreemptive(int n, int at[], int bt[]){
    int wt[MAX]={0}, tat[MAX]={0}, ct[MAX]={0};
    int rt[MAX]={0};
    int completed[MAX]={0};
    int time=0, count=0;

    float total-wt=0, total-tat=0, total-rt=0;

    while (count<n){
        int min=9999, idx=-1;
        for (int i=0; i<n; i++){
            if (at[i]<=time && !completed[i])
            {
                if (bt[i]<min){
                    min=bt[i];
                    idx=i;
                }
            }
        }
        if (idx!=-1){
            rt[idx]= time-at[idx];
            wt[idx]= rt[idx];
            time+= bt[idx];
            ct[idx]= time;
            tat[idx]= ct[idx]-at[idx];
            total-wt += wt[idx];
            total-tat += tat[idx];
            total-rt += rt[idx];
```

```
        completed[idx]=1;
        count++;
    }
    else{
        time++;
    }
}
printf("\n --- SJF Non-preemptive --- \n");
printf("PID \tAT\t BT\t CT\t TAT\t WT\t RT\n");
for (int i=0; i<n; i++){
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
}
```

```
printf("\n --- SJF preemptive (SRTF) ---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i=0; i<n; i++){
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
printf("\nAverage waiting Time = %.2f", total_wt/n);
printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Response Time = %.2f\n", total_rt/n);
}

int main(){
    int n, choice, at[MAX], bt[MAX];
    printf("Enter no of processes:");
    scanf("%d", &n);
    for(int i=0; i<n; i++){
        printf("Enter number of processes");
        scanf("%d", &n);
        for(int i=0; i<n; i++){
            printf("\nprocess %d\n", i+1);
            printf("Arrival Time:");
            scanf("%d", &at[i]);
            printf("Burst Time:");
            scanf("%d", &bt[i]);
        }
        printf("\nChoose Algorithm:");
        printf("\n1.FCFS\n2.SJF Non-preemptive
            \n3.SJF preemptive (SRTF)");
```

```
printf("\nEnter choice:");
scanf("%d", &choice);

switch(choice){
case 1:
    FCFS(n, at, bt);
    break;

case 2:
    SJF_non-preemptive(n, at, bt);
    break;
case 3:
    sjf_preemptive(n, at, bt);
    break;
default:
    printf("Invalid choice");
}
return 0;
}
```

Output:

Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 7

Process 2
Arrival Time: 8
Burst Time: 3

Process 3
Arrival Time: 3
Burst Time: 4

Process 4
Arrival Time: 5
Burst Time: 6

choose Algorithm.

1) FCFS
2) SJF Non-preemptive
3) SJF Preemptive (SRTF)

Enter choice:

--- FCFS ---

PID	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6
P2	8	3	20	2	9	9

Average TAT = 9.75.
Average WT = 4.75
Average RT = 4.75.

Gantt chart:

P1	P3	P4	P2
0	7	11	17	20.

Enter choice:

---- SJF Non-preemptive ---

Process	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P2	3	4	11	8	4	4
P3	5	6	20	15	9	9
P4	8	3	14	6	3	3

Average TAT = 9.00
Avg WT = 4.00
Avg RT = 3.25

Gantt chart

P1	P2	P4	P3
0	7	11	14	20

Enter choice.

---- SJF Preemptive ---

process	AT	BT	CT	TAT	WT	RT.
P1	0	7	7	7	0	0
P2	8	3	11	3	0	0
P3	3	4	14	11	7	4
P4	5	6	20	15	9	9

Avg TAT = 9.00
Avg WT = 4.00
Avg RT = 3.25

8+3
1713.

Week-3.

```
#include<stdio.h>
struct process{
    int pid;
    int bt;
    int at;
    int pr;
    int ct;
    int tat;
    int wt;
    int rt;
    int finished;
};

struct GanttChart {
    int pid;
    int start;
    int end;
};

void printGanttChart(struct GanttChart g[],int count)
{
    printf("\n--- Gantt Chart ---\n");
    printf("\n");
    for(int i=0;i<count;i++){
        int len=g[i].end-g[i].start;
        for(int j=0;j<len-1;j++)
            printf(" ");
        printf("P%d",g[i].pid);
    }
    for(int j=0;j<len-1;j++)
        printf(" ");
    printf("|");
    }
    printf("\n");
    for(int i=0;i<count;i++){
        for(int j=g[i].start;j<g[i].end;j++)
            printf("---");
        printf(" ");
    }
    printf("\n");
    printf("%d",g[0].start);
    for(int i=0;i<count;i++){
        int len=g[i].end-g[i].start;
        for(int j=0;j<len;j++) printf("  ");
        printf("%d",g[i].end);
    }
    printf("\n");
}

void priority_non_preemptive(struct process P[],int n)
{
    int time=0,completed=0;
    struct GanttChart g[100];
    int gcount=0;

    while(completed<n){
        int idx=-1;
        int highest=9999;
        for(int i=0;i<n;i++){
            if(!p[i].finished && p[i].at<=time){
                if(p[i].pr<highest){
                    highest=p[i].pr;
                    idx=i;
                }
            }
        }
    }
```

--- Gantt Chart ---

P1	P2	P3
0	7	10	12

--- Preemptive Priority Scheduling ---

PID	AT	BT	PR	CT	TAT	WT
1	0	7	2	10	10	3
2	2	3	1	5	3	0
3	3	2	3	12	9	7

Avg TAT = 7.33 Avg WT = 3.33

--- Gantt Chart ---

P1	P2	P1	P3
0	2	5	10	12

To simulate CPU Scheduling Algorithm
Round Robin Algorithm.

```
#include <stdio.h>
int main()
{
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int id[n], at[n], bt[n], rt[n];
    int ct[n], tat[n], wt[n], start[n];

    for(int i=0; i<n; i++)
    {
        id[i] = i+1;
        printf("\n Arrival Time for P%d: ", i+1);
        scanf("%d", &at[i]);
        printf("\n Burst time for P%d: ", i+1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
        start[i] = -1;
    }
    printf("\n Enter Time Quantum: ");
    scanf("%d", &tq);

    for(int i=0; i<n-1; i++)
    {
        for(int j=0; j<n-i-1; j++)
        {
            if(at[j] > at[j+1])
            {
```

```
        int temp;
        temp = at[j]; at[j] = at[j+1]; at[j+1] = temp;
        temp = bt[j]; bt[j] = bt[j+1]; bt[j+1] = temp;
        temp = rt[j]; rt[j] = rt[j+1]; rt[j+1] = temp;
        temp = pid[j]; pid[j] = pid[j+1]; pid[j+1] = temp;
      }
    }
  }

  int queue[100], front = 0, rear = 0;
  int visited[n];
  for (int i = 0; i < n; i++)
      visited[i] = 0;

  int time = at[0];
  queue[rear++] = 0;
  visited[0] = 1;

  int gantt P[200], gantt T[200], g = 0;
  float sumTAT = 0, sumWT = 0, sumRT = 0;

  printf("\n -- Round Robin Scheduling --");

  while (front < rear)
  {
    int i = queue[front++];
    if (startt[i] == -1)
        startt[i] = time;
    gantt P[g] = pid[i];
    gantt T[g] = time;
    g++;

    if (rt[i] > tq)
    {
      time += tq;
      rt[i] = tq;
    }
    else
    {
      time += rt[i];
      rt[i] = 0;
      ct[i] = time;
      tat[i] = ct[i] - at[i];
      wt[i] = tat[i] - bt[i];

      sumTAT += tat[i];
      sumWT += wt[i];
      sumRT += (startt[i] - at[i]);
    }

    for (int j = 0; j < n; j++)
    {
      if (at[j] <= time && visited[j] == 0)
      {
        queue[rear++] = j;
        visited[j] = 1;
      }
    }
    if (rt[i] > 0)
    {
      queue[rear++] = i;
    }
    if (front == rear)
    {
```

```
for (int j=0; j<n; j++)
{
    if (visited[j] == 0)
    {
        time = at[j];
        queue[rear++] = j;
        visited[j] = 1;
        break;
    }
}
}
gantt[g] = time;
printf("\n Gantt chart: \n");
for (int i=0; i<g; i++)
    printf("| P%d |", gantt[i]);
printf("\n %d", gantt[0]);
for (int i=1; i<=g; i++)
    printf("  %d", gantt[i]);
printf("\n\n Process | AT | BT | CT | TAT | WT | RT |\n");
for (int i=0; i<n; i++)
{
printf("\n P%d | %d | %d | %d | %d | %d | %d |",
    id[i], at[i], bt[i], ct[i], tat[i], wt[i], start[i] -
    at[i]);
}
printf("\n\n Average TAT: %.2f", sumTAT/n);
printf("\n Average WT: %.2f", sumWT/n);
printf("\n Average RT: %.2f\n", sumRT/n);
return 0;
}
```

Output:-

```
Enter number of processes : 6
Arrival Time for P1: 5
Burst Time for P1: 5
Arrival Time for P2: 4
Burst Time for P2: 6
Arrival Time for P3: 3
Burst Time for P3: 7
Arrival Time for P4: 1
Burst Time for P4: 9
Arrival Time for P5: 2
Burst Time for P5: 2
Arrival Time for P6: 6
Burst Time for P6: 3
Enter Time Quantum : 4
```

Time quantum = 4

Ready queue

Gantt chart:

	P4	P5	P3	P2	P1	P4	P6	P3	P2	P4	P4
0	1	5	9	11	15	19	23	26	29	31	32	33

Process	AT	BT	CT	TAT	WT	RT
P4	1	9	33	32	23	0
P5	2	2	7	5	3	3
P3	3	7	29	26	19	4
P2	4	6	31	27	21	7
P1	5	5	32	27	22	10
P6	6	3	26	20	17	17

```
Average TAT: 22.83
Average WT : 17.50
Average RT : 6.83
```

Week-5

Q) To - Todays - lab execute.

Q) Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. The processes in the system are divided into two categories - system processes & user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

⇒ #include <stdio.h>
#define MAX 100.

typedef struct {
    int id, arrival, burst;
    int completion, turnaround, waiting;
    int type;
    int done;
} Process;

void sortByArrival (Process p[], int n) {
    Process temp;
    for (int i=0; i<n-1; i++) {
        for (int j= i+1; j<n; j++) {
            if (p[i].arrival > p[j].arrival) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

void displayQueues (Process p[], int n, int time) {
    printf ("\n Time %d \n", time);
    printf ("\n System Queue: ");
    int found = 0;
    for (int i=0; i<n; i++) {
        if (!p[i].done && p[i].arrival <= time && p[i].type == 0) {
            printf ("P%d ", p[i].id);
            found = 1;
        }
    }
    if (!found) printf ("Empty");
    printf ("\n User Queue: ");
    found = 0;
    for (int i=0; i<n; i++) {
        if (!p[i].done && p[i].arrival <= time && p[i].type == 1) {
            printf ("P%d ", p[i].id);
            found = 1;
        }
    }
    if (!found) printf ("Empty");
    printf ("\n");
}

int main() {
    int n;
    Process p[MAX];
    printf ("--- CPU scheduling Simulator ---\n");
    printf ("Enter number of processes: ");
    scanf ("%d", &n);

```
for (int i=0; i<n; i++){
    printf("\nProcess %d Id:", i);
    p[i].id = i;

    printf("Arrival Time: ");
    scanf("%d", &p[i].arrival);

    printf("Burst Time: ");
    scanf("%d", &p[i].burst);

    printf("Type (0 = System, 1 = User): ");
    scanf("%d", &p[i].type);
    p[i].done = 0;
}

sort(p, n, Arrival);
int time = 0, completed = 0;

int gantt[MAX], gTime[MAX];
int gindex = 0;
gTime[gindex++] = 0;

while (completed < n){
    display(queue, p, n, time);
    int idx = -1;

    for (int i=0; i<n; i++){
        if (!p[i].done && p[i].arrival <= time && p[i].type==0){
            idx = i;
            break;
        }
    }
}
```

```
if (idx == -1){
    for (int i=0; i<n; i++){
        if (!p[i].done && p[i].arrival <= time && p[i].type == 1){
            idx = i;
            break;
        }
    }
}

if (idx == -1){
    time++;
    continue;
}
printf("Selected: P%d\n", p[idx].id);

gantt[gindex-1] = p[idx].id;
time += p[idx].burst;
gTime[gindex++] = time;

p[idx].completion = time;
p[idx].turnaround = time - p[idx].arrival;
p[idx].waiting = p[idx].turnaround - p[idx].burst;

p[idx].done = 1;
completed++;
}

printf("\nGantt Chart:\n");
for (int i=0; i<gindex-1; i++)
    printf("| P%d ", gantt[i]);
```

Time 9
System Queue: Empty
User Queue: P1 P3.
Selected: P1.

Time 12
System Queue: Empty
User Queue: P3
selected: P3

Gantt Chart:

| P0 | P2 | P1 | P3 |
0 5 9 12 14.

ID	Type	AT	RT	CT	TAT
0	System	0	5	5	5
2	System	1	4	9	8
1	User	2	3	12	10
3	User	3	2	14	11

Average waiting Time: 5.0
Average Turnaround Time:

--- Execution Complete

Week-6.

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate-Monotonic
b) Earliest - deadline First.

```
#include<stdio.h>
#include<math.h>

#define MAX 10

typedef struct {
    int id;
    int burst;
    int deadline;
    int period;
    int completion;
    int waiting;
    int turnaround;
} Process;

void sortEDF(Process p[], int n){
    for(int i=0; i<n-1; i++){
        for(int j=i+1; j<n; j++){
            if(p[i].deadline > p[j].deadline){
                process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```
void printRMS( process p[], int n){
  printf("\n ===== Rate Monotonic Scheduling (RMS) ...
  printGantt(p,n);

printf("| ID |BT| Period |CT|WT|TAT|\n");
  for (int i=0; i<n; i++){
    printf("|%2d| %2d |%6d|%2d|%2d|%2d|\n",
      p[i].id,
      p[i].burst,
      p[i].period,
      p[i].completion,
      p[i].waiting,
      p[i].turnaround);
  }
}

void printGantt (Process p[], int n)
  int time=0;
  printf("\n Gantt Chart \n");
  printf("%d", time);

for (int i=0; i<n; i++){
  printf("| P%d |", p[i].id);
  time += p[i].burst;
  printf("%d", time);
}
printf("\n");
}
```

```
int main(){
  int n;
  Process p[MAX], ed[MAX], rm[MAX];
  printf("Enter number of process: ");
  scanf("%d", &n);

  printf("\n Enter process details \n");

  for (int i=0; i<n; i++){
    p[i].id = i+1;

    printf("\n Process %d: \n", i+1);

    printf("Burst Time: ");
    scanf("%d", &p[i].burst);

    printf("Deadline (EDF): ");
    scanf("%d", &p[i].deadline);

    printf("Period (RMS): ");
    scanf("%d", &p[i].period);
  }

  for (int i=0; i<n; i++){
    ed[i] = p[i];
    rm[i] = p[i];
  }

  sortEDF(ed,n);
  calculateTime(ed,n);
  printEDF(ed,n);
  checkEDF(p,n);

  sortRMS(rm,n);
  calculateTime(rm,n);
  printRMS(rm,n);
  checkRMS(p,n);
  return 0;
}
```

Output:-
Enter number of process : 3

Enter process Details :-
Process 1:
Burst Time : 3
Deadline (EDF) : 10
Period (RMS) : 6.

Process 2:
Burst Time : 2.
Deadline (EDF) : 5.
Period (RMS) : 4.

Process 3:-
Burst Time : 1.
Deadline (EDF) : 8.
Period (RMS) : 3.

====== Earliest Deadline First (EDF) ======

Gantt Chart :-
| P2 | P3 | P1 |
0 2 3 6

| ID | BT | Deadline | CT | WT | TAT |
2 2 5 2 0 2
3 1 8 3 2 3
1 3 10 6 3 6

EDF Utilization = 1.33
Result :- NOT SCHEDUABLE (EDF).

====== Rate Monotonic Scheduling (RMS) ======

Gantt Chart :-
| P3 | P2 | P1 |
0 1 3 6

| ID | BT | Period | CT | WT | TAT |
3 1 3 1 0 1
2 2 4 3 1 3
1 3 6 6 3 6

RMS Utilization = 1.33
RMS Bound = 0.78
Result :- NOT Schedulable (RMS).

For schedulable.
Enter no of process : 3.
Enter process details :-
Process 1:
Burst Time : 3
Deadline (EDF) : 20
Period (RMS) : 20.

Process 2:-
Burst Time : 2
Deadline (EDF) : 5
Period (RMS) : 5.

ID	BT	Deadline	CT	WT	TAT
2	2	5	2	0	2
3	2	10	4	2	4
1	3	20	7	4	7

EDF Utilization = 0.75.

Result :- Schedulable (EDF).

=== Rate Monotonic Scheduling (RMS) ===

ID	BT	Period	CT	WT	TAT
2	2	5	2	0	2
3	2	10	4	2	4
1	3	20	7	4	7

RMS Utilization = 0.75

RMS Bound = 0.78

Result :- SCHEDULABLE (RMS)

3|3

Q) The following program consists of 3 concurrent processes & 3 binary semaphores. The semaphores are initialized as S0 = 1, S1 = 0, S2 = 0. Find out how many time process P0 will print "0". Assume the order of execution as P0, P1, P2, P0, P1.

Process P0:
```
while (true) {
    wait(S0);
    printf("0");
    Signal(S1);
    Signal(S2);
}
```

Process P1:
```
wait(S1);
Signal(S0);
```

Process P2:
```
wait(S2);
Signal(S0);
```

Sol:-

Initial	S0 = 1	S1 = 0	S2 = 0	Print
P0	0	1	1	0
P1	1	0	1	-
P2	1	0	10	-
P0	0	1	1	0
P1	1	0	1	-

The P0 will print 0 two times.

Q) Consider two processes A & B. Two semaphore variable S & T are used to synchronize the process. S is initialized to 0 & T is initialized to 1. Processes are scheduled in the following order A B A B B A A B A. What will be printed on the screen?

```
Process A:
While(1)
{
    wait(S);
    print 'P';
    print 'P';
    Signal (T);
}
```

Process	S=0	T=1	Print
A	0	1	Process A is bl
B	1	0	IT
A	0	1	IT PP
B	1	0	IT PP IT
B	1	0	Process B is blo
A	0	1	IT PP IT PP
A	0	1	Process A is blo
B	1	0	IT PP IT PP IT
A	0	1	IT PP IT PP IT P

Write a C program to simulate
a) Producer - Consumer problem using semaphore.
b) Dining - Philosophers problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define MAX_ITEMS 15

int buffer[BUFFER_SIZE];
int in=0;
int out=0;

sem_t empty;
sem_t full;
sem_t mutex;

int produce_item(int i){
    return i;
}

void consumer_item(int item, int index){
    printf("Consumed %d from buffer[%d]\n", item, index);
}

void* producer(void* arg){
    int i, index, item;
```

Output:-

Produced: 0 at buffer [0]
Consumed: 0 from buffer [0]
Produced: 1 at buffer [1]
Consumed: 1 from buffer [1]
Produced: 2 at buffer [2]
Produced: 3 at buffer [3]
Consumed: 2 from buffer [2]
Produced: 4 at buffer [4]
Consumed: 3 from buffer [3]
Produced: 5 at buffer [0]
Consumed: 4 from buffer [4]
Produced: 6 at buffer [1]
Produced: 7 at buffer [2]
Consumed: 5 from buffer [0]
Produced: 8 at buffer [3]
Consumed: 6 from buffer [1]
Produced: 9 at buffer [4]
Produced: 10 at buffer [0]
Consumed: 7 from buffer [2]
Produced: 11 at buffer [1]
Consumed: 8 from buffer [3]
Produced: 12 at buffer [2]
Consumed: 9 from buffer [4]
Produced: 13 at buffer [3]
Produced: 14 at buffer [4]
Consumed: 10 from buffer [0]
Consumed: 11 from buffer [1]
Consumed: 12 from buffer [2]
Consumed: 13 from buffer [3]
Consumed: 14 from buffer [4]

Table:-

Buffer size = 5
Buffer = [_, _, _, _, _]
in = 0, out = 0
empty = 5, full = 0, mutex = 1

Step	Operation	Buffer state	in	out	empty	full	mutex
1	produces	[0, _, _, _, _]	1	0	4	1	1
	inserts 0						
2)	Consumes remove 0	[_, _, _, _, _]	1	1	5	0	1
3)	Produces insert 1	[_, 1, _, _, _]	2	1	4	1	1
4)	Consumed remove 1	[_, _, _, _, _]	2	2	5	20	1
5)	Produces insert 2	[_, _, 2, _, _]	3	2	4	1	1
6)	produces insert 3	[_, _, 2, 3, _]	4	2	3	2	1
7)	Consumes remove 2	[_, _, _, 3, _]	4	3	4	1	1
8)	produces insert 4	[_, _, _, 3, 4]	5	3	3	2	1
9)	Consumes remove 3	[_, _, _, _, 4]	5	4	4	1	1
10)	Consumes insert 5	[5, _, _, _, 4]	6	4	3	2	1

b) Dining - Philosophers problem.

→
```
# include <stdio.h>
# include <pthread.h>
# include <semaphore.h>
# include <unistd.h>

# define N 3
# define MAX_ROUNDS 3

sem_t chopstick[N];
sem_t room;

void think(int i){
    printf("Philosopher %d is thinking\n", i);
    sleep(1);
}

void eat(int i){
    printf("Philosopher %d is eating\n", i);
    sleep(1);
}

void *philosopher(void *num){
    int i = *(int *)num;

    for(int j=0; j<MAX_ROUNDS; j++){
        think(i);
        sem_wait(&room);

        sem_wait(&chopstick[i]);
        printf("Philosopher %d picked up fork %d\n", i, i);
        sem_wait(&chopstick[(i+1)%N]);
        printf("Philosopher %d picked up fork %d\n", i, (i+1)%N);

        eat(i);

        sem_post(&chopstick[i]);
        sem_post(&chopstick[(i+1)%N]);
        printf("Philosopher %d put down fork %d and %d\n", i, i, (i+1)%N);
        sem_post(&room);
    }
    return NULL;
}

int main(){
    pthread_t phil[N];
    int phil_num[N];

    for(int i=0; i<N; i++){
        sem_init(&chopstick[i], 0, 1);
        phil_num[i] = i;
    }

    sem_init(&room, 0, N-1);

    for(int i=0; i<N; i++)
        pthread_create(&phil[i], NULL, philosopher, &phil_num[i]);
```

Output:-

philosopher 2 is thinking.
philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 0 picked up right fork 1.
philosopher 2 picked up right fork 0.
philosopher 2 picked up left fork 2.
philosopher 2 is eating.
philosopher 2 put down forks 2 & 0.
philosopher 2 is thinking.
philosopher 0 picked up left fork 0.
philosopher 0 is eating.
philosopher 1 picked up right fork 2.
philosopher 0 put down forks 0 & 1.
philosopher 0 is thinking
philosopher 1 picked up left fork 1.
philosopher 1 is eating.
philosopher 2 picked up right fork 0.
philosopher 1 put down forks 1 & 2.
philosopher 2 picked up left fork 2.
philosopher 2 is eating.
philosopher 0 picked up right fork 1.
philospher 1 is thinking.
philosopher 2 put down forks 2 & 0.
philosopher 0 picked up left fork 0.
philosopher 0 is eating.
philosopher 1 picked up right fork 2.
philosopher 2 is thinking.
philosopher 0 put down forks 0 & 1.
philosopher 1 picked up left fork 1.
philosopher 1 is eating.

Output:-

philosopher 3 is thinking
philosopher 0 is thinking.
philosopher 2 picked up right fork 0
philosopher 1 put down forks 1 & 2.
philosopher 1 is thinking.
philosopher 2 picked up left fork 2.
philosopher 2 is eating.

Table			
Step	operation	Fork state	Room.
1	P3 enters	[-,-,-,-,-]	3
2	P0 enters	[-,-,-,-,-]	2
3	P1 enters		

Write a program to simulate:

a) Banker's algorithm for the purpose of deadlock avoidance.

⇒
```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10

int main() {
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
    int available[MAX];

    printf("Enter Allocation Matrix:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Maximum Demand Matrix:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    int work[MAX], safeSeq[MAX];
    bool finish[MAX];

    for (int i=0; i<m; i++)
        work[i] = available[i];

    for (int i=0; i<n; i++)
        finish[i] = false;

    int count=0;

    while (count<n) {
        bool found = false;

        for (int i=0; i<n; i++) {
            if (!finish[i]) {
                bool possible = true;

                for (int j=0; j<m; j++) {
                    if (need[i][j] > work[j]) {
                        possible = false;
                        break;
                    }
                }
```

```
if (possible)
  for (int j=0; j<m; j++) {
    work[j] += alloc[i][j];
  }

safeSeq[count++] = i;
finish[i] = true;
found = true;
  }
}

if (!found) {
  printf("\n System is NOT in a safe state (Deadlock possible)\n");
  return 0;
}

printf("\n System is in a safe state.\n");
printf("\n Safe sequence is: ");

for (int i=0; i<n; i++) {
  printf("P%d", safeSeq[i]);
  if (i != n-1)
    printf(" -> ");
}

printf("\n");
return 0;
}
```

Output:

```
Enter number of processes: 5
Enter number of resources: 4
Enter Allocation Matrix:
0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4

Enter maximum Demand Matrix:
0 0 1 2
1 7 5 0
2 3 5 6
0 6 5 2
0 6 5 6

Enter Available Resources:
1 5 2 0

System is in a safe state.
Safe sequence is: P0 -> P2 -> P3 -> P4 -> P1
```

Bankers Algorithm with resource Allocation added

```
int process;
int request[MAX];
printf("\n Enter process number making request: ");
scanf("%d", &process);

printf("Enter request vector: \n");
for (int i=0; i<m; i++)
    scanf("%d", &request[i]);

for (int i=0; i<m; i++){
    if(request[i] > need[process][i]){
    printf("\n Error: Request exceeds maximum need\n");
    return 0;
}
}

for (int i=0; i<m; i++){
    if (request[i] > available[i]){
    printf("\n Resources not available. Process must wait\n");
    return 0;
}
}

for (int i=0; i<m; i++){
    available[i] = request[i];
    alloc[process][i] += request[i];
    need[process][i] -= request[i];
}

printf("\n Request can be
```

O/P:

```
Enter process number: 1
Enter request:
0 2 1 0
Request Granted.
```

Bankers Algorithm Detection.

```
#include <stdio.h>
#include<stdbool.h>
#define MAX 10

int main(){
    int n, m;
printf("Enter number of process: ");
scanf("%d", &n);

printf("Enter number of resources: ");
scanf("%d", &m);
int alloc[MAX][MAX], request[MAX][MAX];
int available[MAX];

printf("Enter Allocation Matrix: \n");
for (int i=0; i<n; i++){
    for (int j=0; j<m; j++)
        scanf("%d", &alloc[i][j]);

printf("Enter Request Matrix: \n");
    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++)
        scanf("%d", &available request[i][j]);
```

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst fit.

b) Best fit

c) First fit.

```
#include<stdio.h>
#define MAX 100

void allocate(int blocks[], int m, int processes[], int n, int type){
int allocation[MAX];

for(int i=0; i<n; i++)
    allocation[i] = -1;

for(int i=0; i<n; i++){
    int idx = -1;
    for(int j=0; j<m; j++){
        if(block[j] >= process[i]){
            if(type == 1){
                idx = j;
                break;
            }
        else if(type == 2){
            if(idx == -1 || blocks[j] < blocks[idx])
                idx = j;
        }
    else if(type == 3){
        if(idx == -1 || block[j] > blocks[idx])
            idx = j;
        }
    }
}
```

```
        if (idx != -1) {
            allocation[i] = idx;
            blocks[idx] -= processes[i];
        }
    }

    if (type == 1)
        printf("\n First Fit Allocation:\n");
    else if (type == 2)
        printf("\n Best Fit Allocation:\n");
    else
        printf("\n Worst Fit Allocation:\n");

    printf("Process\tSize\tBlock\n");

    for (int i=0; i<n; i++) {
        printf("P%d\t%d\t", i+1, processes[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}

int main() {
    int blocks[MAX], p[MAX];
    int b1[MAX], b2[MAX], b3[MAX];
    int m, n;

    printf("Enter number of blocks:");
    scanf("%d", &m);

    printf("Enter block size:\n");
    for (int i=0; i<m; i++) {
        scanf("%d", &blocks[i]);

        b1[i] = blocks[i];
        b2[i] = blocks[i];
        b3[i] = blocks[i];
    }

    printf("Enter number of processes:");
    scanf("%d", &n);

    printf("Enter process sizes:\n");
    for (int i=0; i<n; i++)
        scanf("%d", &p[i]);

    allocate(b1, m, p, n, 1);
    allocate(b2, m, p, n, 2);
    allocate(b3, m, p, n, 3);

    return 0;
}
```

Output:

```
Enter number of blocks: 5
Enter block sizes: 400 700 200 300 600
Enter number of processes: 4
Enter process sizes: 212 512 312 526

First Fit Allocation:
```

Process	Size	Block
P1	212	1
P2	512	2
P3	312	5
P4	526	Not allocated

Best Fit Allocation:

Process	Size	Block
P1	212	4
P2	512	5
P3	312	1
P4	526	2

Worst Fit Allocation:

Process	Size	Block
P1	212	2
P2	512	5
P3	312	2
P4	526	Not Allocated

Write a C program to simulate page replacement algorithm.
a) FIFO
b) LRU
c) Optimal

```
#include<stdio.h>
#include<stdlib.h>

int findOptimal(int pages[], int n, int frame[], int frameCount)
    int farthest = index, pos = -1;
    for (int i=0; i<frameCount; i++){

int isPresent(int frame[], int n, int page)
{
    for (int i=0; i<n; i++)
        {
        if (frames[i] == page)
            return i;
        }
    return 0;
}
void FIFO(int pages[], int n, int capacity)
{
  int frames[MAX], index=0, faults=0;
for (int i=0; i<capacity; i++)
        frames[i] = -1;
 printf("In FIFO page replacement \n");
  for (int i=0; i<n; i++)
  {
    if (!isPresent(frames, capacity, pages[i]))
    {
    frames[index] = pages[i];
```

```
printf("Enter frame capacity: ");
scanf("%d", &capacity);

FIFO(pages, n, capacity);
LRU(pages, n, capacity);
Optimal(pages, n, capacity);

return 0;
}
```

Output:

```
Enter number of pages: 20
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter frame capacity: 3
FIFO Page faults: 15
LRU Page faults: 12
Optimal Page faults: 9
```

FIFO page Replacement

Page 7 → 7 - -
Page 0 → 7 0 -
Page 1 → 7 0 1
Page 2 → 2 0 1
Page 0 → 2 0 1
Page 3 → 2 3 1
Page 0 → 2 3 0
Page 4 → 4 3 0
Page 2 → 4 2 0
Page 3 → 4 2 3
Page 0 → 0 2 3
Page 3 → 0 2 3

Page 2 → 0 2 3
Page 1 → 0 1 3
Page 2 → 0 1 2
Page 0 → 0 1 2
Page 1 → 0 1 2
Page 7 → 7 1 2
Page 0 → 7 0 2
Page 1 → 7 0 1
Total Page Faults = 15

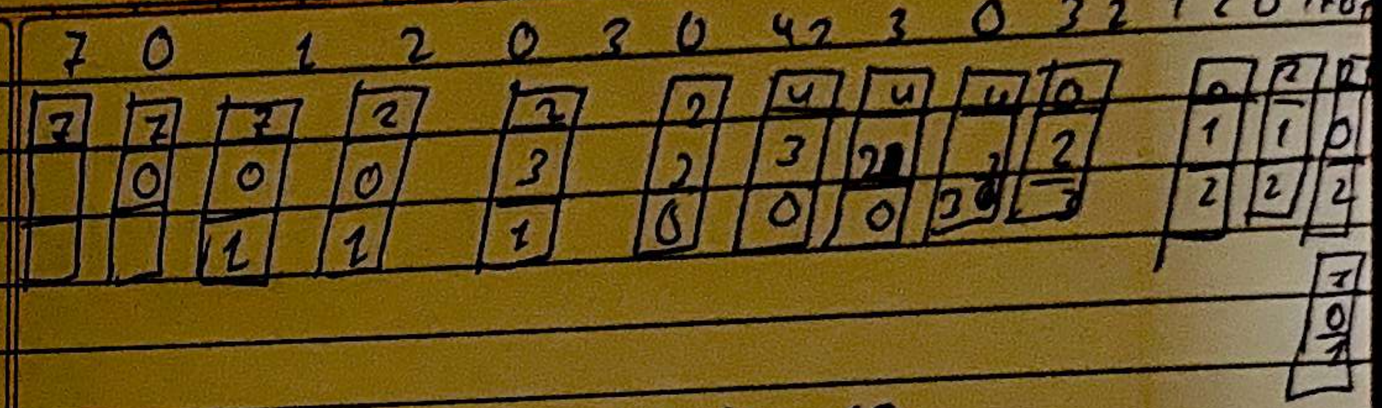
LRU page ~~faults~~ Replacement

Page 7 → 7 - -
Page 0 → 7 0 -
Page 1 → 7 0 1
Page 2 → 2 0 1
Page 0 → 2 0 1
Page 3 → 2 0 3
Page 0 → 2 0 3
Page 4 → 4 0 3
Page 2 → 4 0 2
Page 3 → 4 3 2
Page 0 → 0 3 2
Page 3 → 4 3 2
Page 2 → 0 3 2
Page 1 → 1 3 2
Page 2 → 1 2 2
Page 0 → 1 0 2
Page 1 → 1 0 2
Page 7 → 1 0 7
Page 0 → 1 0 7
Page 1 → 1 0 7
Total Page faults = 12

Optimal Page Replacement.

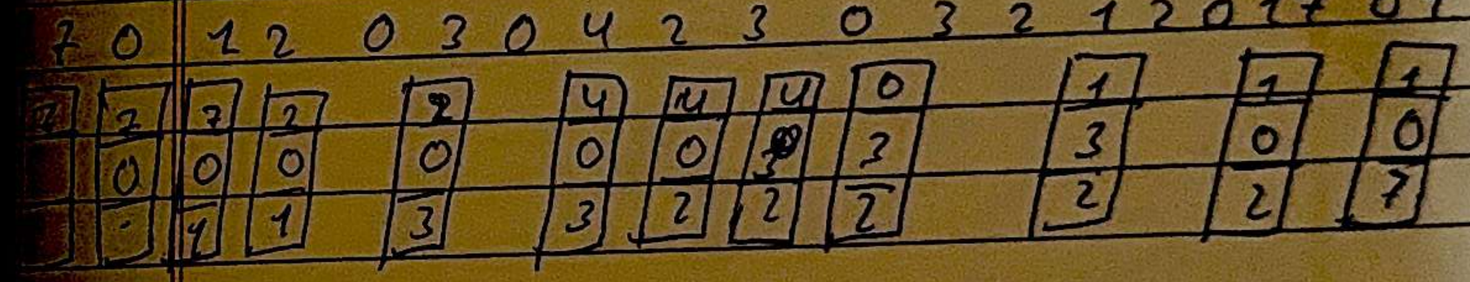
Page 7 → 7 - -
Page 0 → 7 0 -
Page 1 → 7 0 1
Page 2 → 2 0 1
Page 0 → 2 0 1
Page 3 → 2 0 3
Page 0 → 2 0 3
Page 4 → 2 4 3
Page 2 → 2 4 3
Page 3 → 2 4 3
Page 0 → 2 0 3
Page 3 → 2 0 3
Page 2 → 2 0 3
Page 1 → 2 0 1
Page 20 → 2 0 1
Page 0 → 2 0 1
Page 1 → 2 0 1
Page 7 → 7 0 1
Page 0 → 7 0 1
Page 1 → 7 0 1

1) FIFO.



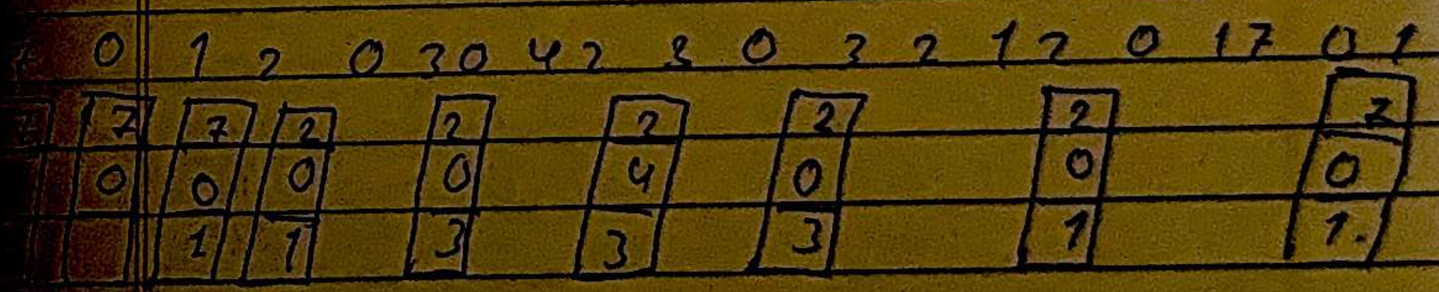
Total No of faults = 15.

2) LRU - least Recently Used.



Total No of page faults = 12.

3) Optimal.



Total No of page faults = 9.